Программная инженерия

Лекция 3: Тестирование ПО

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Зачем нужны тесты?

Самый частый ответ

- Контроль качества и корректности работы программы
- Проверка, что ничего не сломали после коммита
- Подтверждение ожидаемого поведения системы
- Страховочная сетка для разработчиков

Недостаточно думать только так

- Тесты делают больше, чем просто ловят баги

Тесты - это документация требований

Что это?

- Исполняемая документация требований
- Тест формулирует, как система должна себя вести
- Тесты живут рядом с кодом и запускаются автоматически

Зачем это нужно?

- Меньше двусмысленности, чем в текстовых документах
- Актуальность: зелёные тесты — требование выполняется
- Про поддержку тестов забыть **сложнее**, чем про поддержку документации

Тест как спецификация в коде

Ключевые характеристики спецификации

- Явно задаёт условия и ожидаемый результат
- Проверяемое поведение описано в исполняемом виде
- Несоответствие сразу проявляется падением теста
- Тесты эволюционируют вместе с системой

Хороший тест — спецификация в виде кода

Тесты и уверенность в продукте

Тесты дают устойчивую уверенность в продукте

- Тесты поддерживают быстрый темп разработки

Зачем это нужно?

- Рефакторинг становится менее рискованным
- Новые фичи и фиксы не ломают старый функционал

Роли тестов

Ключевые роли

1. Выявляют дефекты
2. Документируют требования
3. Дают уверенность при рефакторинге

Важно

- Тестирование вшивается в жизненный цикл, а не идёт “после”

SWEBOK: Карта раздела по тестированию

Что должен знать специалист про тесты по мнению SWEBOK?

Пять ключевых блоков

1. **Foundations**: базовые понятия, цели, ограничения
2. **Test Levels**: уровни тестирования от unit до acceptance
3. **Test Techniques**: black-box, white-box, experience-based
4. **Test Process**: планирование, выполнение, отчетность
5. **Tools & Automation**: фреймворки, системы управления тестами

Тестирование по SWEBOK: зачем?

Определение

- Тестирование — деятельность по оценке качества продукта и его улучшению за счёт выявления дефектов и проблем
- Динамическая проверка того, что программа предоставляет **ожидаемое** поведение на **конечном** наборе тестовых случаев

Ограничения тестирования по SWEBOK

Что нельзя ожидать от тестов

- Исчерпывающее тестирование для реальных систем **нереализуемо**
- Тесты **не доказывают** корректность программы
- Тесты только **снижают** неопределённость поведения

Ключевые проблемы

- Проектирование хороших тест-кейсов и критерии достаточности
- Oracle problem (знание верного результата)
- Масштабируемость и автоматизация тестирования

Группы техник тестирования в SWEBOK

Три семейства техник

- Specification-based (black-box) техники
- Structure-based (white-box) техники
- Experience-based техники
- Рекомендация: комбинировать техники, а не выбирать одну

Зачем нужны техники

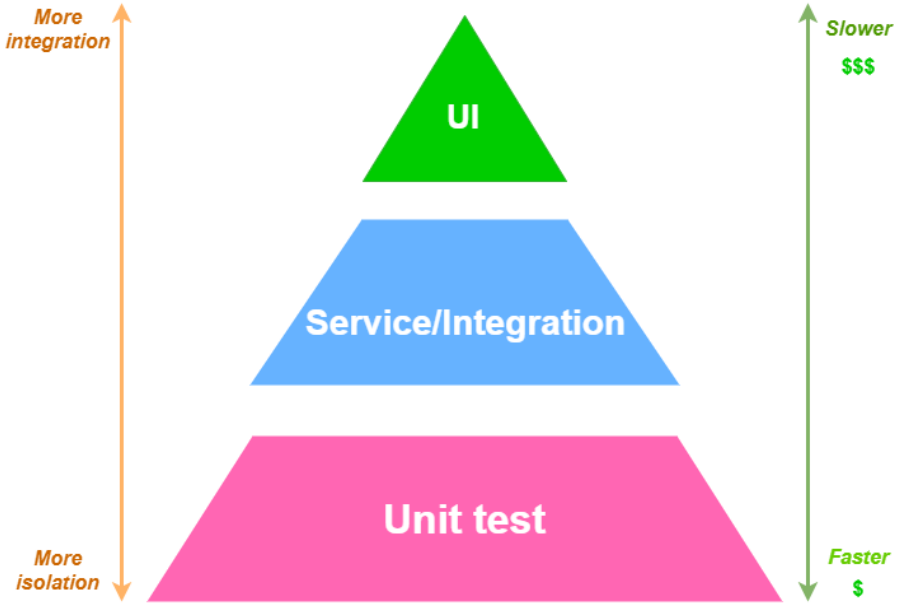
- Систематизируют проектирование тест-кейсов
- Позволяют рационально использовать ограниченное время

SWEBOK про тестирование. Резюме

Главные тезисы

- Тестирование — *систематическая, измеримая* деятельность. Набор активностей с разными масштабами и целями
- На нижних уровнях проверяется *корректная реализация* проектных решений, на верхних — *соответствие* системы требованиям и ожиданиям заказчика

Тестовая пирамида



Уровни пирамиды

- **Unit-тесты**: быстрые, изолированные, проверка функций/классов
- **Интеграционные**: взаимодействие модулей (например, сервис + БД)
- **E2E**: путь пользователя через весь стек

Почему пирамида?

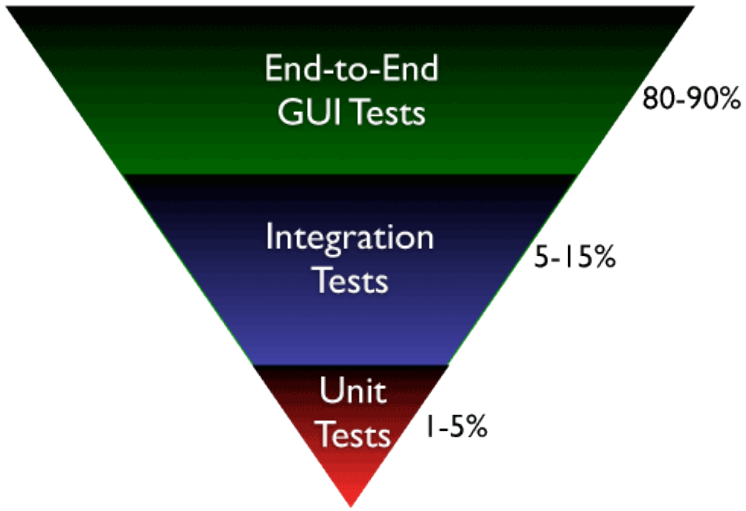
Баланс скорости, надёжности и стоимости

- Unit-тесты: секунды, легко писать/править
- Интеграционные: минуты, уверенность в связях
- E2E: часы, хрупкие, дорогая поддержка

Цель

- Много дешёвых тестов внизу для частой проверки
- Мало дорогих наверху для ключевых сценариев
- Чем ниже в пирамиде обнаружилась - ошибка, тем дешевле её исправить

Антипаттерн: перевёрнутая пирамида



Проблемы

- Медленные тесты
- CI/CD тормозит
- Сложно и дорого поддерживать

Правильные пропорции

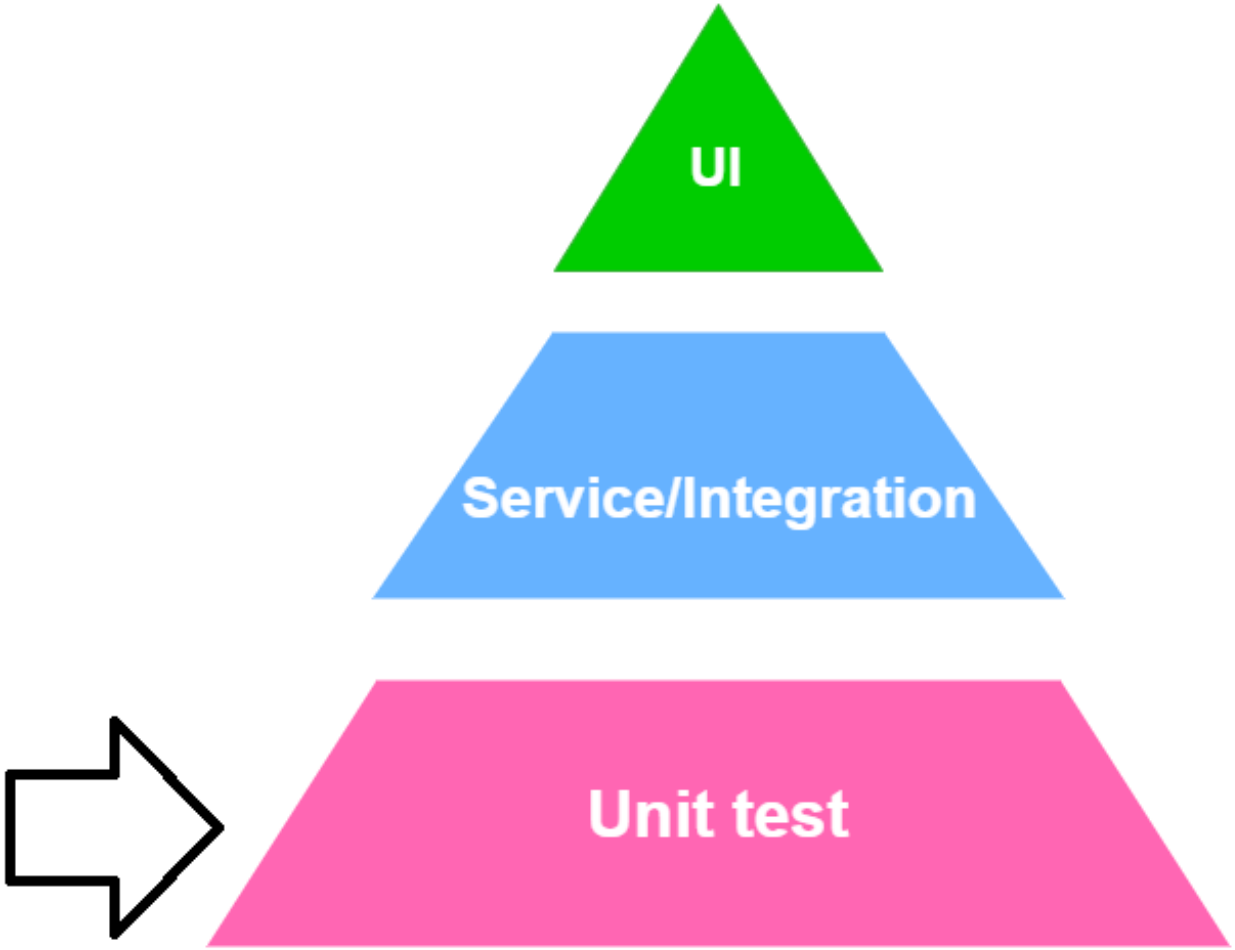
Рекомендации

- **70%** unit-тестов
- **20%** интеграционных
- **10%** E2E

Адаптация под проект

- Микросервисы: акцент на интеграцию
- Монолит: акцент на юниты

Unit тесты



Основание пирамиды тестирования

Unit тесты

Определение

- Проверка маленького изолированного фрагмента кода (функции/метода) в отрыве от системы

Ключевые характеристики

- Гарантирует предсказуемое поведение логики при заданных входных данных

Unit тесты: Пример

```python
import unittest

class TestSum(unittest.TestCase):
    def test_list_int(self):

        data = [2, 4, 6]
        result = sum(data)
        self.assertEqual(result, 12)

if __name__ == '__main__':
    unittest.main()
```

Unit тест на python

Зачем разработчику unit-тесты?

Преимущества

- Ловят ошибки на ранней стадии, до интеграции и ручного тестирования
- Ускоряют отладку: падение теста сразу указывает на проблемный код
- Уменьшают страх менять код и рефакторить

Unit-тесты и качество архитектуры

Влияние на код

- Тестируемость требует **чистого и слабо связанного** кода
- × Сложный, сильно связанный код трудно тестировать — сигнал к рефакторингу

Как это работает?

- Unit-тесты можно и **нужно** запускать в пайплайне CI
- Регулярный прогон тестов создает страховку для изменений. Старый функционал (скорее всего) не сломается

Тестовый код так же важен, как продакшн код

Unit-тесты тоже можно написать плохо

- Уделяйте тестам столько же внимания, сколько production-коду
- Нет оправданиям “это всего лишь тесты” для плохого кода

Рекомендации по составлению тестов

- Один тест – одно условие
- Используйте структуру: “arrange, act, assert” или “given, when, then”
- Не переусердствуйте с DRY: дублирование OK, если улучшает читаемость (DRY vs DAMP)

Антипаттерны Unit-тестов

Полезные материалы

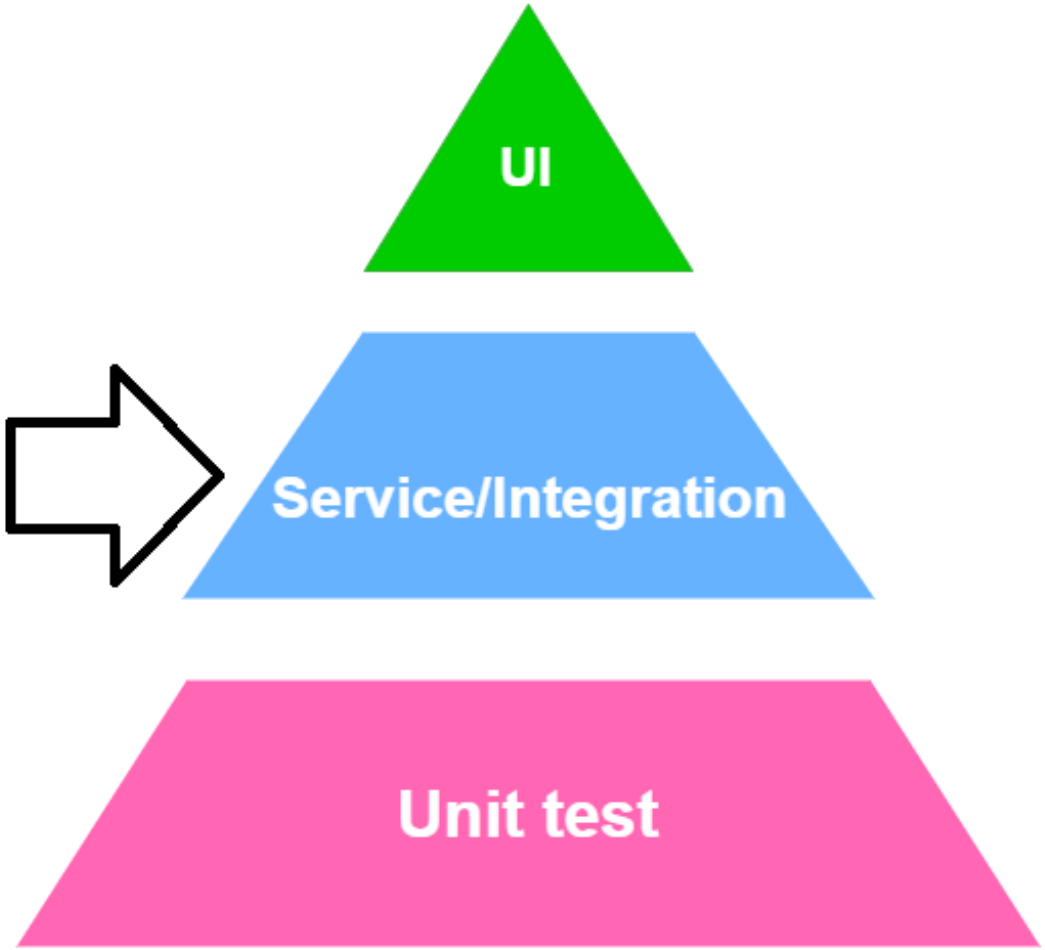
- `https://www.yegor256.com/2018/12/11/unit-testing-anti-patterns.html`
- “xUnit Test Patterns” by Gerard Meszaros

Итог: Unit тесты

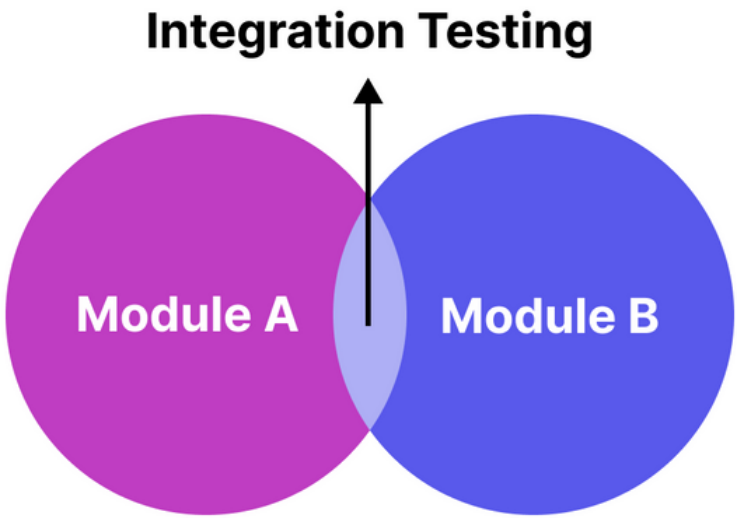
Главные тезисы

- Проверяет изолированные фрагменты кода на корректность
- Ловит ошибки на ранних этапах, ускоряет отладку
- Способствует созданию чистой и слабо связанной архитектуры
- Автоматизация через фреймворки и CI повышает надёжность кода
- Окупается за счёт снижения регрессий и уверенности при рефакторинге

Интеграционные тесты



Интеграционные тесты



Зачем это нужно?

- Убедиться, что модули/сервисы работают корректно **вместе**
- Выявить ошибки на **стыках** компонентов (форматы данных, протоколы, маршрутизация)
- Проверить **цепочки взаимодействия**: от HTTP-запроса до записи в базу

Уровни интеграционных тестов

Типы интеграции

- **Модульная**: внутри одного приложения (контроллер + сервис + база)
- **Системная**: связка подсистем (фронтенд + бэкенд, платёжный сервис + биллинг)
- **С внешними API**: проверка клиента против провайдера (Google Maps, Weather)

Узкие vs широкие интеграционные тесты

Узкие тесты

- Фокус на **коде вашего компонента**
- Используют **тестовые дублёры** (mock)
- Быстрые, многочисленные, подходят для **ранних стадий CI/CD**

Широкие тесты

- Требуют **живых сервисов** и полноценных окружений
- Прогоняют потоки через **несколько систем**
- Медленные, хрупкие, для **smoke-тестов** или поздних этапов

Стратегии интеграции

Классические подходы

- **Top-down**: сверху вниз (Заглушки для низких модулей)
- **Bottom-up**: снизу вверх (Заглушки для верхних модулей)
- **Incremental**: по частям (гибрид, для CI/CD)

Top-down интеграция

Описание

- Тестирование **сверху вниз**: UI/контроллеры первыми
- Заглушки заменяют нижние модули (БД, сервисы)
- Имитируют ответы: stub БД возвращает тестовый JSON

Плюсы/минусы

- + Раннее покрытие бизнес-логики и UI
- - Много заглушек, нижние ошибки поздно

Bottom-up интеграция

Описание

- Тестирование снизу вверх: БД/утилиты первыми
- Заглушки вызывают нижние модули как клиенты
- Driver симулирует контроллер, передавая входные данные

Плюсы/минусы

- + Ранние базовые ошибки, параллельная работа
- - Системные фичи тестируются поздно

Incremental интеграция

Описание

- Гибрид: поэтапная сборка кластеров модулей
- Комбинирует top-down + bottom-up подходы
- Пример: сначала сервис+БД, потом +API контроллер

Плюсы/минусы

- + Fail-fast в CI/CD, минимум заглушек
- - Требует планирования зависимостей

Проблемы интеграционных тестов

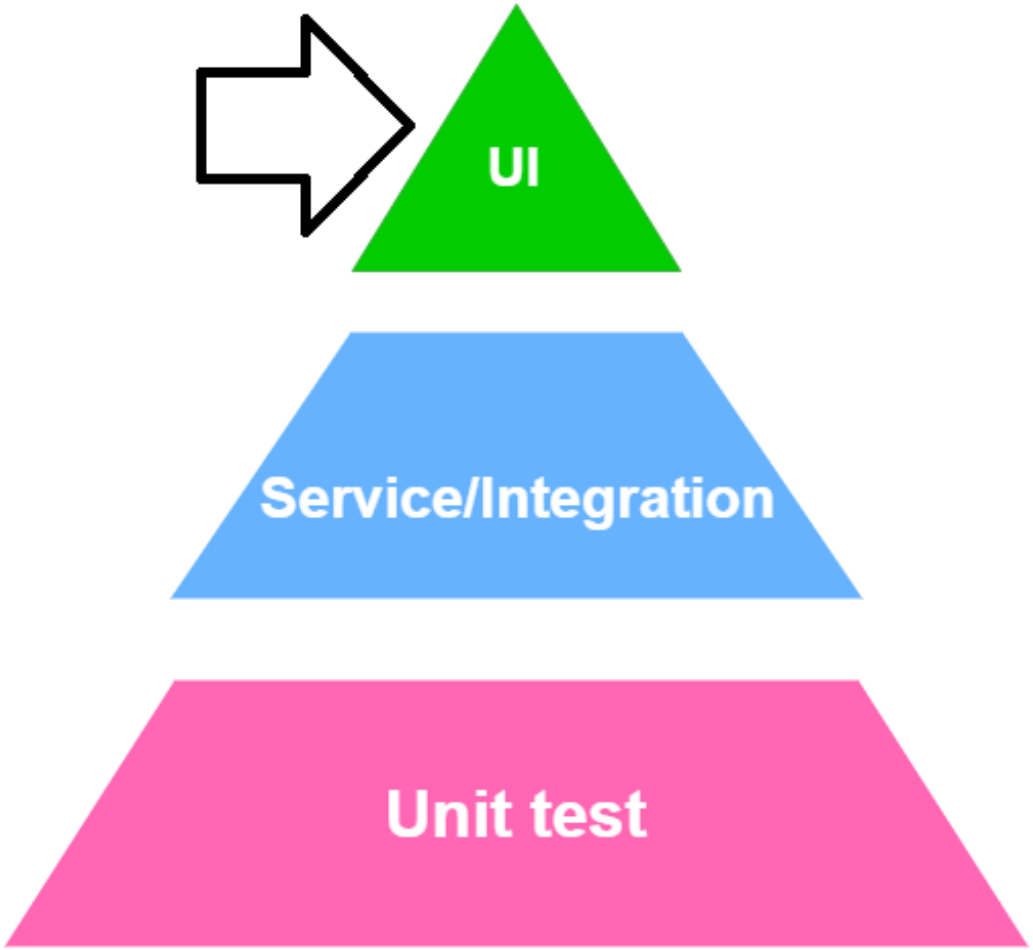
Типичные проблемы

- Флейковые тесты: нестабильные результаты (сеть, гонки, конфиги)
- Тестовые окружения: дорого, нестабильно
- Потеря доверия: разработчики игнорируют красные сборки

Решения

- Узкие тесты с дублёрами для **стабильности**
- Широкие тесты только для **критичных путей**
- Мониторинг в продакшене для **дополнительной уверенности**

End-to-End тесты



Вершина пирамиды тестирования

End-to-End тесты: суть и роль

Что такое E2E тесты?

- Проверяют ключевые пользовательские сценарии через все слои системы
- Симулируют реальные действия пользователя (регистрация, оформление заказа)
- Проходят через UI, бизнес-логику, интеграции и базу данных

Зачем они нужны?

- Дают максимальную уверенность в целостности системы
- Находят проблемы, недоступные для юнит- и интеграционных тестов

Автоматизация UI: как это работает

Процесс автоматизации

- Тест взаимодействует с приложением как браузер или мобильный клиент
- Открывает страницы, вводит данные, кликает по элементам
- Проверяет состояние UI и соответствие ожидаемому результату

Проблемы E2E тестов

Медлительность

- Запуск занимает десятки минут
- Используют реальные данные и окружения
- Замедляют CI/CD

Флейки (flaky) тесты

- Падают/проходят без изменений кода
- Причины: нестабильные селекторы, гонки

Правила проектирования E2E тестов

Короткие сценарии

- 3–5 шагов максимум (открыть → ввести → кликнуть → проверить)
- Длинные тесты (>10 мин) флейкают из-за накопления состояний

Моки внешних зависимостей

- Заменяйте 3rd-party API на моки
- Исключает сетевые фейлы и недоступность сервисов

Итог: E2E тесты

Главные тезисы

- E2E тесты проверяют пользовательские сценарии через все слои системы
- Требуют борьбы с медлительностью и флейками
- Проектируйте короткие сценарии и используйте моки

Мокеры

```
from unittest import mock
import pytest

@mock.patch('get_data_from_redis')
def test_getImages(mock_get_redis):
    mock_get_redis.return_value = {'id':1, 'name':'Mount
    res = getImages() # method for which we are writing
    assert res.status_code == 200
    mock_get_redis.assert_called_once()
```

Заменяют функционал модуля “заглушкой”

Mockers в unit-тестах

Действия

- Заменяют реальные зависимости
- Эмулируют ответы внешних компонентов системы
- Позволяют контролировать поведение окружения теста

Зачем это нужно?

- Изолировать логику unit от инфраструктуры
- Делать тесты быстрыми и детерминированными

Что именно mockers изолируют в unit-тестах

Типичные зависимости

- Databases и in-memory хранилища
- Message brokers и очереди
- External APIs и сторонние сервисы
- File system

Эффект

- Нет сетевых вызовов и I/O в unit-тестах
- Минимум нестабильности окружения

Mockers в integration-тестах

Что это?

- Подменяют внешние границы системы
- Оставляют реальные взаимодействия между внутренними модулями

Зачем это нужно?

- Тестировать интеграции без внешних рисков
- Сохранять стабильность тестов

Что mockers эмулируют в integration-тестах

Внешние системы

- Third-party сервисы и SaaS-платформы
- Payment gateways и billing-системы

Сценарии поведения

- Успешные ответы с реалистичными данными
- Таймауты, ошибки и нестандартные коды

Глубина использования mockers

Unit-тесты

- Мокается большинство зависимостей
- Фокус на одной функции или классе
- Почти нет реальных внешних вызовов

Integration-тесты

- Мокается только внешний контур
- Несколько реальных компонентов вместе
- Проверка wiring и data flow

Другие виды тестирования

Что ещё существует?

- **Smoke Tests** — базовая проверка работоспособности
- **Performance/Load/Stress** — оценка под нагрузкой
- **Acceptance Tests** — соответствие требованиям заказчика
- **System vs Functional** — интеграция и отдельные функции
- **Non-Functional** — безопасность, удобство, совместимость
- **А также:** Regression, Security, Usability, Compatibility, Localization, …

Метрики тестов

Зачем измерять?

- Оценка качества тестирования
- Поддержка принятия решений о готовности ПО

Что рассмотрим?

- Code Coverage
- Modified Condition/Decision Coverage (MC/DC)
- Метрики эффективности: DDP, Test Case Efficiency, Defect Density

Line coverage

Определение

- Доля исполненных строк кода во время тестов
- Формула: выполненные строки / все строки

Ограничения

- Одна строка может скрывать несколько ветвлений
- 100% не гарантирует проверки альтернативных путей
- Только sanity-check, не критерий достаточности

Код с подсветкой строк

```
1 reference
35  public static bool OnlyDigitsInRange(string text, int range_left_included, int range_right_included)
36  {
37      if (string.IsNullOrWhiteSpace(text))
38      return false;
39
40      if (!text.All(char.IsDigit))
41          return false;
42
43      if (int.TryParse(text, out int number))
44      {
45          return number >= range_left_included && number <= range_right_included;
46      }
47      return false;
48  }
49
```

Ключевые моменты

- Подсветка выполненных строк во время тестов
- Визуализация линейного покрытия
- Показывает только факт исполнения строк

Покрытие ветвей (Branch Coverage)

Что это?

- Все исходы решений: true/false для if/switch
- Формула: выполненные ветви / все ветви
- Для одного if — две ветви (true/false)

Преимущества

- Лучше коррелирует с реальным поведением
- Заставляет продумывать все входные данные
- Выявляет дефекты в edge-cases

Modified Condition/Decision Coverage (MC/DC)

Определение

- MC/DC — формализованный критерий достаточности тестирования
- Каждое условие должно независимо влиять на итог решения

Ключевые характеристики

- Стандарт для критичных систем (DO-178C, ISO 26262)
- Требование: два теста для каждого условия с изменением результата
- Повышает вероятность нахождения дефектов в логике безопасности

Тесты для MC/DC

True	False	Source & Details	multicondition.c All Files
5		1	`int multicondition(int a, int b, int c, int d){`
1	4	2	`  if ((a || b) && (c || d)){`
0			1 `(T || _) && (T || _)`
0			2 `(T || _) && (F || T)`
0			3 `(F || T) && (T || _)`
1			4 `(F || T) && (F || T)`
	0		5 `(T || _) && (F || F)`
	1		6 `(F || T) && (F || F)`
	3		7 `(F || F) && (_ || _)`
-			MC/DC a: 1, 7 2, 7
+			MC/DC b: 3, 7 4, 7
-			MC/DC c: 1, 5 3, 6
+			MC/DC d: 2, 5 4, 6

Каждое условие меняет результат независимо

Интерпретация метрик покрытия

Что показывает покрытие?

- Высокое покрытие **не значит** отсутствие багов
- Тесты могут проверять только happy path
- Баги могут быть в нефункциональных аспектах

Сколько должно быть тестов?

Ответ

- Нет универсального числа
- Зависит от задачи и рисков

Главное

- Количество тестов — грубая метрика
- Важна плотность тестов вокруг рискованных областей

Резюме. Метрики тестов

Главные тезисы

- Покрытие кода — инструмент управления рисками, не KPI
- Branch coverage лучше line coverage для выявления дефектов
- MC/DC — строгий стандарт для критичных систем

Связь автотестов и экономики

Автотесты — это не только про качество кода, но и про экономику проекта

Кому выгодны тесты?

- Разработчикам — быстрее писать код и безопасно его менять
- Работодателю — защита вложений в кодовую базу и уже *оплаченные* фичи

Код как актив бизнеса

Инвестиция работодателя

- Бизнес платит не за строки кода, а за фичи и багфиксы, дошедшие до продакшена
- Программисты — промежуточный этап, конечный актив — кодовая база
- Задача тестов — защитить этот актив от деградации при изменениях

Код без тестов как ремонт машины без гарантии

Пример

- Вы чините машину и ожидаете гарантию на отремонтированную деталь
- Готовы платить за новые поломки, но не за ту же самую неисправность
- Работодатель также ожидает, что уже исправленный код не сломается снова
- Автотесты — это “гарантийный талон” на уже оплаченный функционал

Автотесты как гарантия

Что именно они гарантируют?

- То, что работало вчера, продолжит работать после изменений
- Покрытие тестами снижает риск того, что бизнес заплатит дважды за одну и ту же фичу

Правило для pull request

Норма команды

- Каждый вклад в кодовую базу должен сопровождаться поддерживающим тестом
- Нет теста — нет объективного доказательства, что изменение безопасно
- Простое правило: “No tests — no merge”

Testing vs Debugging

«Unit testing is a technique that completely replaces debugging.
If debugging is required, the design is bad.»

— *Yegor Bugayenko, "Are You Still Debugging?"*

Debugging

Что такое Debugging?

- Debugging — реактивное исследование уже проявившейся проблемы через debugger, логирование и ручной анализ

Проблема Debugging

- Баг исправили - результат не сохранился
- Без гарантии от повторного появления того же бага

Testing vs Debugging

Testing

- Улучшает дизайн: тестируемый код проще и чище
- Легко повторить: failing test — формальное описание бага

Debugging

- Терпит плохой дизайн: «как-нибудь разберёмся в debugger»
- Слабо повторяемо: ручная работа программистов (дорого), локальные эксперименты

Источники информации и изображений

1. https://www.yegor256.com/2025/06/08/pull-request-without-test.html
2. https://anymindgroup.com/news/tech-blog/15053/
3. https://blog.xnsio.com/2011/02/01/inverting-the-testing-pyramid/
4. https://www.turing.com/kb/unit-testing-in-python
5. https://katalon.com/resources-center/blog/integration-testing
6. https://martinfowler.com/articles/practical-test-pyramid.html
7. https://www.verifysoft.com/HTML-Help-CTC-v10.0.1/source/MCDC_Coverage.html
8. https://www.tothenew.com/blog/mocking-vs-monkey-patching-in-python/

Программная инженерия

Лекция 3: Тестирование ПО

Денисов Артём Владимирович

Уральский Федеральный Университет

2026